FEU INSTITUTE OF TECHNOLOGY

Project Title:

"Health Monitoring System with Pulse Rate and Body Temperature Sensors, SMS Alerts, and LCD Display"

Group Members:

Bañas, Mikail Andrei

Pineda, Yheztin Razel

Rapanut, Renz Danniel

Abstract:

This project involves designing a health monitoring system using an Arduino Uno, which includes several sensors and modules for measuring and reporting critical health statistics. The system includes a Pulse Sensor to track heart rate, an MLX90614 infrared temperature sensor to measure body temperature, and an LCD I2C display for real-time data presentation. A GSM module is also incorporated, which will send SMS notifications when key heart rate and temperature criteria are surpassed. This small and portable device intends to provide continuous health monitoring and instant notifications, so improving personal health management and early detection of any health issues. The device is powered by a 9V battery, ensuring portability and convenience of usage.

General Objective

This project aims to design a portable heart rate monitor that displays the user's heart rate on an LCD screen, detects the user body temperature and sends an SMS notification if the pulse rate falls outside a pre-defined range.

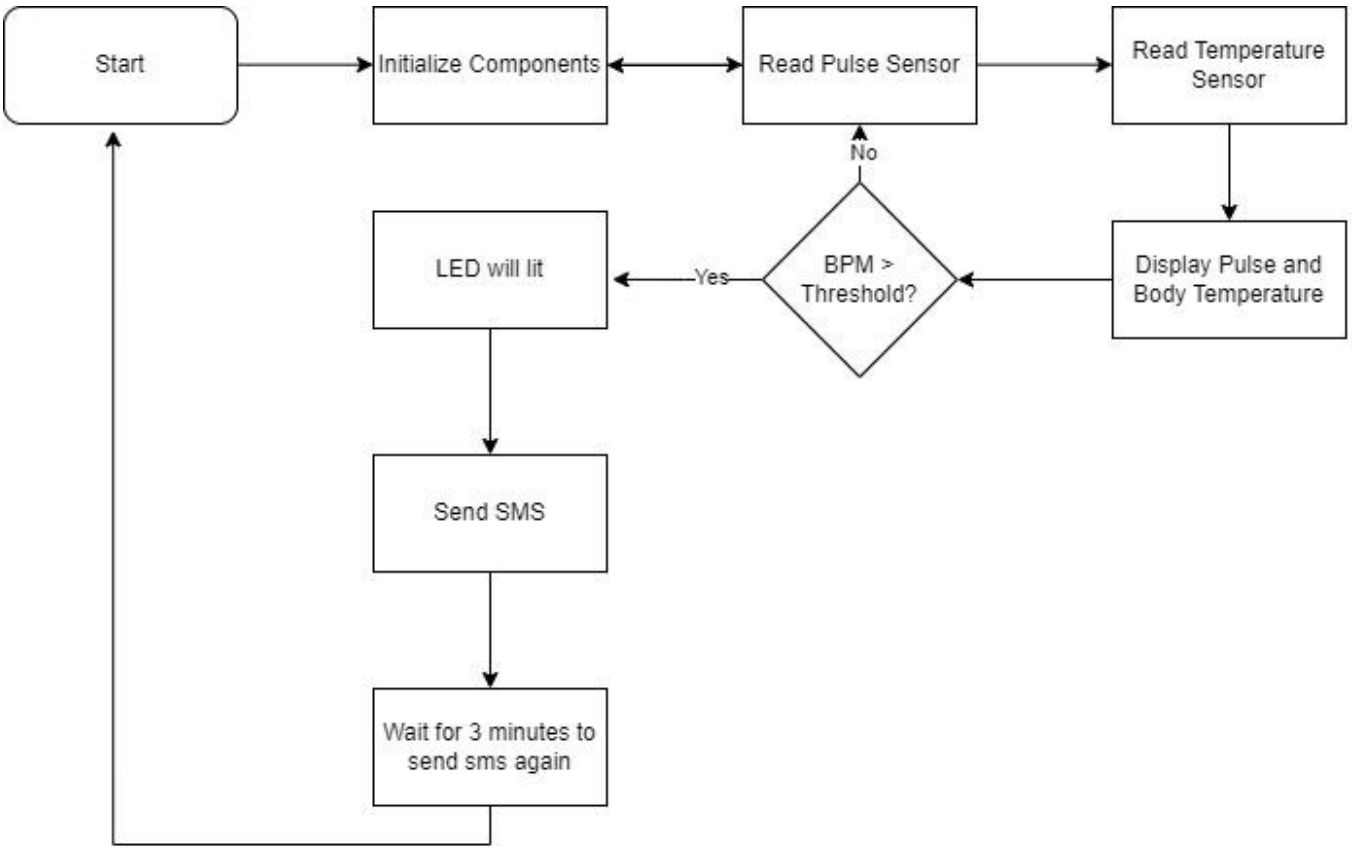
Specific Objectives

- Design a circuit that utilizes a heart rate sensor to detect the user's pulse.
- Develop software to process the sensor data and calculate the heart rate.
- Implement a function to display the heart rate on an LCD screen.
- Integrate SMS functionality to send alerts when heart rate reaches critical levels.
- Utilize a infrared sensor to detect the body temperature of the user.
- Encase the components in a portable and user-friendly housing.

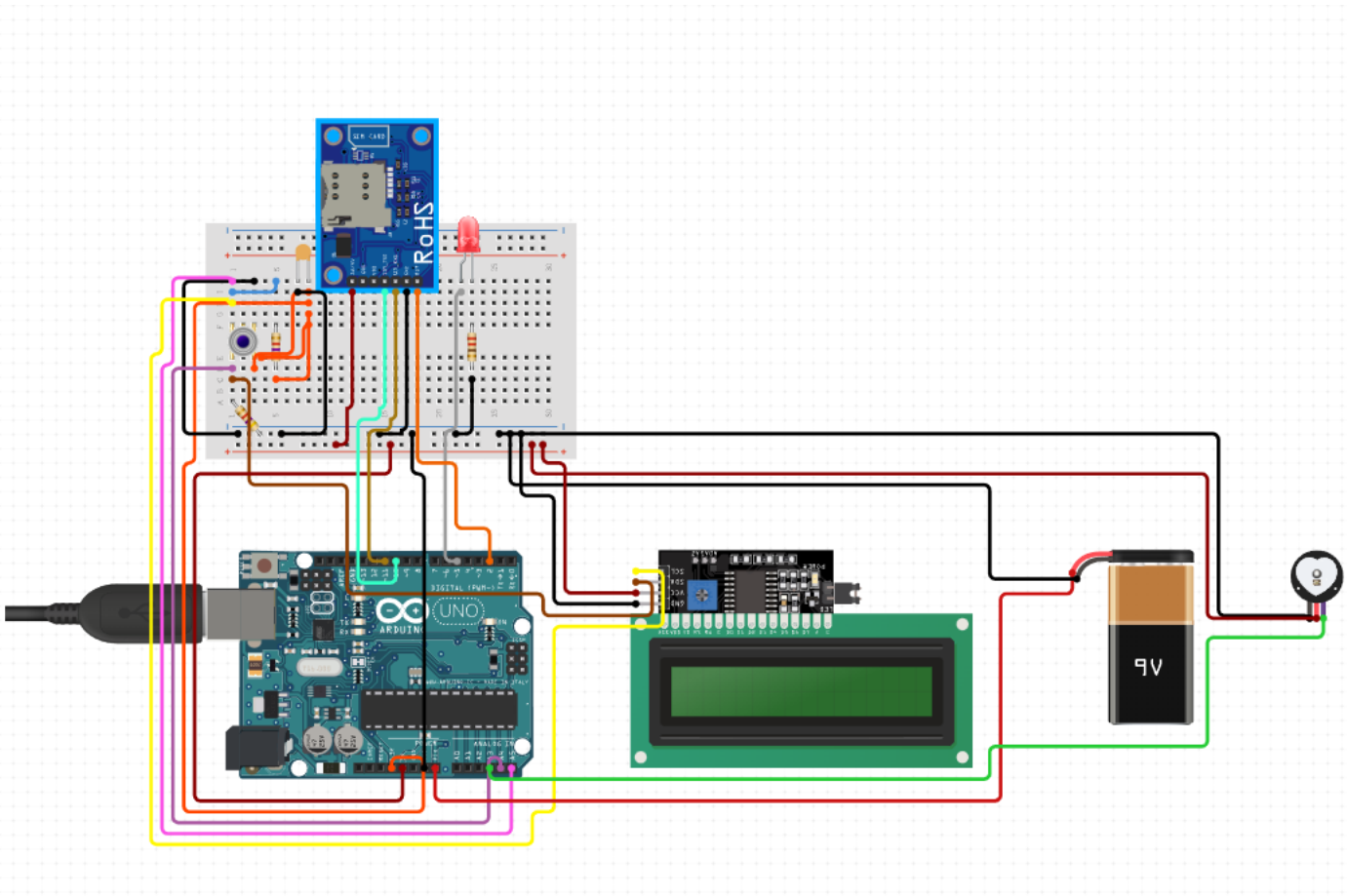
Materials

- Microcontroller board (e.g., Arduino Uno)
- Heart rate sensor (e.g., MAX30102)
- LCD display (e.g., 16x2 character LCD)
- GSM/GPRS module with SIM card (for SMS functionality)
- Jumper wires
- Breadboard (for prototyping)
- Battery source (e.g., Li-ion battery)
- Project enclosure
- LEDs
- MLX90614 infrared Sensors

Flowchart:


Start
Initialize Components
Read Pulse Sensor
Read Temperature Sensor
No
LED will lit
Yes
BPM > Threshold?
Display Pulse and Body Temperature
Send SMS
Wait for 3 minutes to send sms again

Circuit Diagram:



Observation

During the development and testing of the Arduino-based health monitoring system, several key observations were made. The Pulse Sensor accurately detected heartbeats and provided real-time BPM readings, which were displayed on the LCD along with the temperature readings from the MLX90614 sensor. The high BPM LED indicator effectively turned on when the BPM exceeded the set threshold of 100 BPM. The GSM module successfully sent SMS alerts when the BPM exceeded the threshold, provided the interval time had passed. The system maintained consistent performance under continuous operation, with all components functioning as expected.

Conclusion

The Arduino-based health monitoring system successfully integrated a Pulse Sensor, MLX90614 temperature sensor, LCD display, and GSM module to monitor and report vital health parameters. The system effectively detected heart rate and body temperature, displayed the data in real-time, and sent SMS alerts when the BPM exceeded the predefined threshold. The use of a 9V battery provided adequate power for portable operation. Overall, the system demonstrated reliable performance and could serve as a practical tool for continuous health monitoring and immediate alerting.

Source Code:

```
// Include necessary libraries

#define USE_ARDUINO_INTERRUPTS true

#include <PulseSensorPlayground.h>

#include <LiquidCrystal_I2C.h>

#include <Wire.h>

#include <Adafruit_MLX90614.h>

#include <SoftwareSerial.h>


// Initialize LCD

LiquidCrystal_I2C lcd(0x27, 16, 2); // set the LCD address to 0x27 for a 16 chars and 2 line display


// Initialize MLX90614

Adafruit_MLX90614 mlx = Adafruit_MLX90614();


// Constants

const int PULSE_SENSOR_PIN = 0;  // Analog PIN where the PulseSensor is connected

const int LED_PIN = 9;          // On-board LED PIN for pulse blink

const int HIGH_BPM_LED_PIN = 10; // LED PIN for indicating high BPM

const int THRESHOLD = 550;      // Threshold for detecting a heartbeat

const int HIGH_BPM_THRESHOLD = 100; // BPM threshold for high BPM LED
```

```
// GSM module pins
const int RX_PIN = 7;
const int TX_PIN = 8;

// Create PulseSensorPlayground object
PulseSensorPlayground pulseSensor;
SoftwareSerial gsm(RX_PIN, TX_PIN);

// Variables for timing
unsigned long previousMillis = 0;
const unsigned long interval = 180000; // 3 minutes in milliseconds

void setup() {
  // Initialize Serial Monitor
  Serial.begin(9600);
  gsm.begin(9600);
  lcd.init();
  lcd.backlight();

  // Configure PulseSensor
  pulseSensor.analogInput(PULSE_SENSOR_PIN);
  pulseSensor.blinkOnPulse(LED_PIN);
  pulseSensor.setThreshold(THRESHOLD);
```

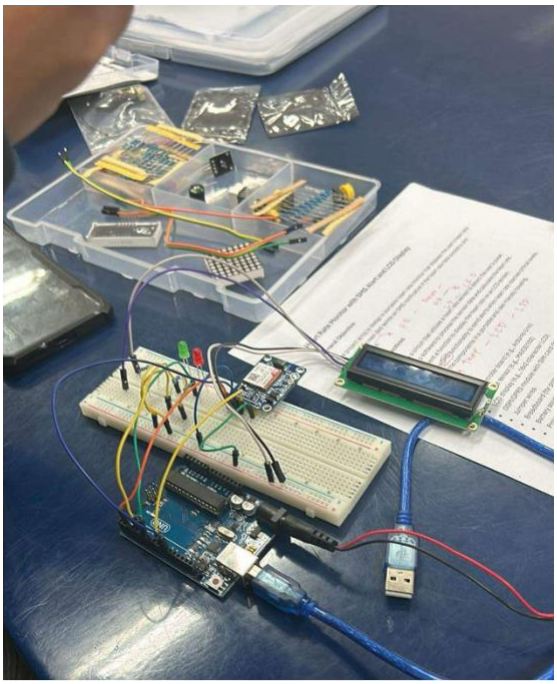
```
  // Check if PulseSensor is initialized
  if (pulseSensor.begin()) {
    Serial.println("PulseSensor object created successfully!");
  }

  // Initialize MLX90614
  if (!mlx.begin()) {
    Serial.println("Error connecting to MLX90614 sensor. Check wiring.");
    while (1);
  }

  // Configure high BPM LED
  pinMode(HIGH_BPM_LED_PIN, OUTPUT);
  digitalWrite(HIGH_BPM_LED_PIN, LOW); // Ensure the LED is off initially
}

void sendSMS(String message) {
  gsm.print("AT+CMGF=1\r"); // Set SMS to text mode
  delay(100);
  gsm.print("AT+CMGS=\"+1234567890\"\r"); //
  delay(100);
  gsm.print(message);
```

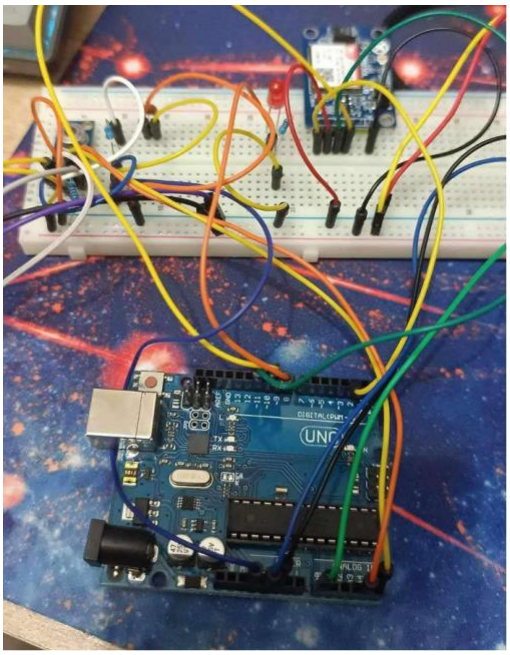
```
  delay(100);

  gsm.write(26); // ASCII code of CTRL+Z to send the message

  delay(1000);

}


void loop() {

  // Get the current Beats Per Minute (BPM)

  int currentBPM = pulseSensor.getBeatsPerMinute();


  // Get the temperature in Celsius

  double temperature = mlx.readObjectTempC();


  // Check if a heartbeat is detected

  if (pulseSensor.sawStartOfBeat()) {

    Serial.println("A HeartBeat Happened!");

    Serial.print("BPM: ");

    Serial.println(currentBPM);


    // Display BPM

    lcd.clear();

    lcd.setCursor(0, 0);

    lcd.print("BPM:");

    lcd.setCursor(5, 0);
```

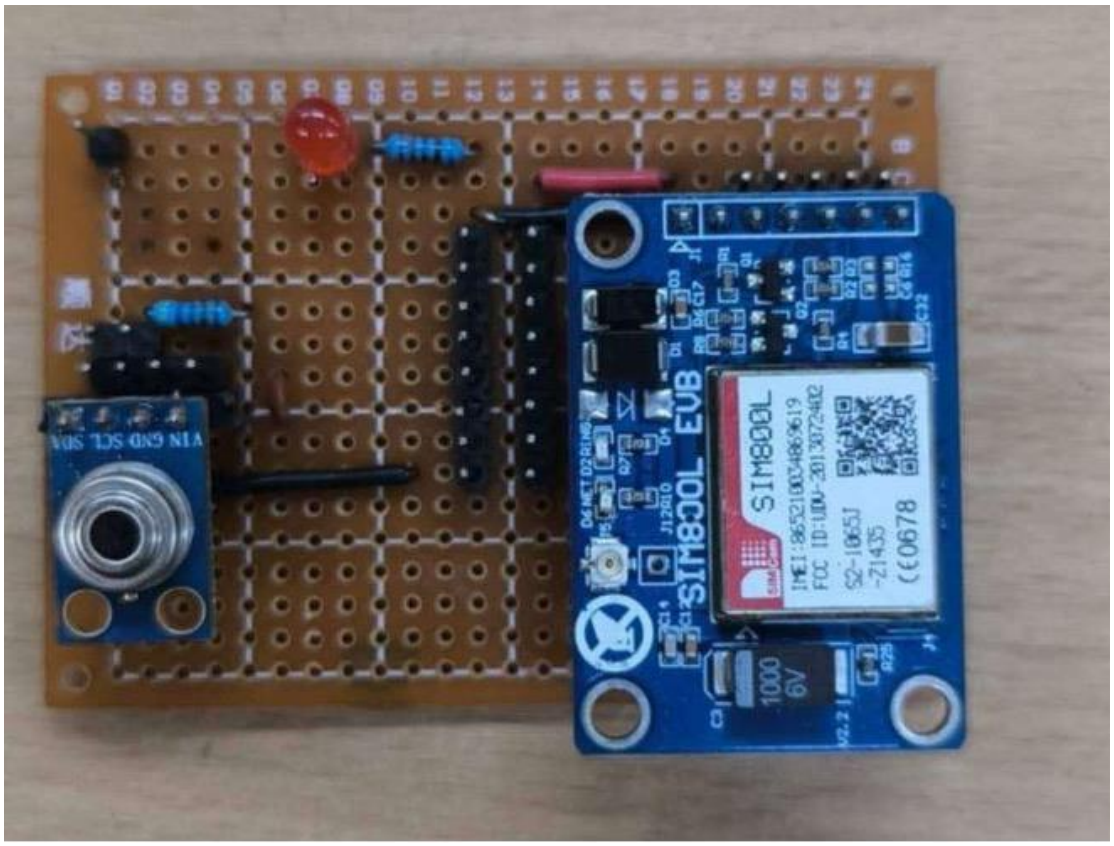
```
  lcd.print(currentBPM);


  // Check if BPM is higher than the threshold
  if (currentBPM > HIGH_BPM_THRESHOLD) {
    digitalWrite(HIGH_BPM_LED_PIN, HIGH); // Turn on the high BPM LED


    // Check if it's time to send a message
    unsigned long currentMillis = millis();
    if (currentMillis - previousMillis >= interval) {
      previousMillis = currentMillis;
      sendSMS("Warning: High BPM detected!");
    }
  } else {
    digitalWrite(HIGH_BPM_LED_PIN, LOW); // Turn off the high BPM LED
  }
}


// Display temperature
lcd.setCursor(0, 1);
lcd.print("Temp: ");
lcd.print(temperature);
lcd.print(" C");
```

```
  // Add a small delay to reduce CPU usage

  delay(1000);

}
```

Documentation






SIM800L EVB
SIM800L
1000
6V
VIN GND SCL SDA